THE OPEN SOURCE AUDIT

A Capstone Project Report for NGMC Course

Subject: Linux Kernel

Course: Open Source Software | Unit Coverage: 1 - 5

Student Name:	*Aditya Talreja*
Registration Number:	*24BCE10891*
Slot:	*E11*
Date of Submission:	*31st March, 2026*

1. Introduction

This audit dives into the Linux Kernel - a core piece behind one of history's most impactful software creations. Inside it lives control over hardware, task timing, memory flow, linking each app to the physical system beneath.

Back in 1991, one young coder from Finland began tinkering - Linus Torvalds, just twenty-one, chasing a dream of open computing. What emerged wasn't merely code, but legacy. Instead of fading, his work grew quietly behind scenes, running much of today's digital backbone. From smartphones to machines on another planet, its presence spreads without fanfare. Though born in simplicity, it now operates where speed and reliability matter most.

This deep look begins at the core's roots, moves through the rights granted by its GPL v2 terms, touches on moral questions around shared code, traces how deeply it lives inside Linux machines, follows the world that grew around it - then lines it up beside closed systems such as those from Redmond and Cupertino. Five small programs tagged along show real command-line work, mirroring values like clear process, repeatable tasks, working together.

Part A - Origin and Philosophy (Units 1 & 2)

A1. Why the Linux Kernel Was Created

Back then, computer operating systems usually came with a high price tag and strict ownership rules. Though Unix packed serious power, it also carried a steep cost. On the flip side, MINIX offered freedom but only ran

on older 16-bit machines. It wasn't built for daily tasks - just classroom lessons.
Even though Richard Stallman's GNU Project built vital software pieces, it didn't have a working core. By 1991, a young Linus Torvalds - only 21 - grew tired of using MINIX on his Intel 386 machine, so he began coding a system center of his own. He shared news about it through a message posted to the comp.os.minix forum, which sparked interest across continents. People from different countries joined in after that announcement. When what Torvalds made was linked with the collection from GNU, complete versions of Linux came into existence.
A single project sparked something unexpected: people everywhere pitching in, making code strong enough to stand beside giants. What began quietly soon challenged old beliefs about who gets to create tech. Not money, but shared effort became its backbone. This shift felt quiet at first - then impossible to ignore.

A2. The Licence – What It Actually Says

Under GPL v2, Linux comes with four key rights built in:

1. Run the software for any purpose.
2. Look at it closely. Then change what needs changing.
3. Redistribute copies.
4. Share improvements publicly.

Freedom sticks around in every changed version, thanks to GPL v2's rules. When firms share altered software, out comes the source code - no exceptions there. Using it privately? That stays off the table for sharing demands.

On the flip, MIT-style rules let someone turn code into private software. GPL v2 pushes back by keeping work open for everyone, yet MIT spreads faster. Picking one spills your belief: strict sharing keeps control in the group, looser terms fuel wide use.

Beer without cost or freedom? Linux offers both using freely and changing openly - this difference powers things like Android plus speeds up fixing flaws. A key split, yet it runs deeper than price tags ever could.

A3. The Ethics of Open Source

What if every program had unlocked code? Sharing code freely tends to speed progress, tighten defenses, and rarely falters on credibility. Systems such as Linux machines, site hosts, storage back ends often grow stronger when opened up. Still, closed programs sometimes spark funding where shared efforts struggle to hold attention.

Who gets left behind when businesses build on shared work? Firms such as Red Hat or Google give something in return, yet others take freely while offering little. Using open source fairly means putting real effort into it - through coding help, money, or time from developers. What matters is balance, not just taking.

From tiny tweaks to big leaps, progress often begins where others left off. Built on common ground, new ideas take shape without redoing what already works. Shared code feeds invention, quietly running the tech behind phones, servers, and more. What grows from open effort moves faster, reaching further because it stands together.

Part B – Linux Footprint (Unit 2)

Here you'll find an explanation of how the Linux Kernel lives inside a Linux setup, focusing mostly on Fedora (RPM), though some details apply to Debian or Ubuntu too.

- ❖ Installation

The kernel is installed and updated via package managers:

RPM-based (Fedora, RHEL, CentOS):

- sudo dnf install kernel # install latest kernel
- sudo dnf update kernel # update kernel

Debian/Ubuntu:

- sudo apt install linux-image-generic
- sudo apt upgrade # updates all packages including kernel

Building from source gives developers room to tweak - grab the code at kernel.org, set options using make menuconfig, build it step by step, installation follows after compilation finishes.

Key Directories

- ❖ /boot – Kernel images (vmlinuz-*), initramfs, and GRUB configs.
- ❖ /proc – Virtual filesystem exposing runtime system info (/proc/cpuinfo, /proc/meminfo).
- ❖ /sys – Virtual filesystem for hardware info; used by udev.
- ❖ /lib/modules/$(uname -r) – Kernel modules for current version.
- ❖ /usr/src/linux-headers-* – Headers for compiling external modules.
- ❖ /etc/modprobe.d – Module load/blacklist configurations.

Privileges and Security

Running deep inside the machine, the kernel holds full control through ring 0, setting who can do what across users and groups. Instead of relying on "and," connections shift - services tied to the kernel, such as udevd or klogd, operate either under root or restricted accounts, reducing exposure. Because changes touch critical areas like /boot and GRUB, only those with root access may update the kernel. Trust matters most; these updates must arrive solely from verified sources. High stakes live in small details, where permission levels shape security from the start.

Useful Commands

- uname -r # current kernel version
- uname -a # full system info
- sudo modprobe <mod> # load module
- sudo modprobe -r <mod> # unload module
- lsmod # list loaded modules
- dmesg # kernel log messages
- sudo grub2-mkconfig -o /boot/grub2/grub.cfg # update GRUB

Update Model

Every few months, a new version of the core Linux system shows up online, guided by Linus Torvalds himself. When problems pop up after release, smaller corrections follow - like 6.6.1 - to handle flaws or weak spots. These adjustments travel through official channels only after public discussion among developers. Instead of hidden code changes, debates happen openly, mostly on email threads anyone can see. Linux distros then gather those verified pieces, bundle them quietly into updates you install automatically. While some operating systems keep fixes behind walls, here the process stays visible to all who care to look.

Part C - The FOSS Ecosystem (Units 3 & 4)

Dependencies: What Linux Relies On

Most of the Linux Kernel uses C code, though certain parts rely on assembly language where hardware demands it. Building it requires specific tools and libraries, which are essential for compiling its components correctly

- GCC/LLVM – Compilers for the C source code.
- GNU Make – Orchestrates the compilation of thousands of files.
- GNU Binutils – Assembler, linker, and object file tools.
- Perl & Python – Scripts and code generation.
- OpenSSL – Signs kernel modules to ensure only trusted modules load.

What Linux Enables

- Android – Over 3 billion devices run a Linux-based OS maintained by Google.
- Linux distributions – Ubuntu, Fedora, Debian, Arch, and hundreds more, each offering unique package management, desktops, and philosophies.
- Chrome OS – Lightweight OS for Chromebooks.
- Containers – Features like namespaces and cgroups enable Docker, Kubernetes, and cloud-native infrastructure.
- Embedded systems – Routers, smart TVs, industrial controllers, medical devices, the ISS, and the Mars Ingenuity helicopter.
- Supercomputers – All of the world's 500 fastest supercomputers run Linux.

Linux and the LAMP stack

Running quietly beneath the surface, Linux powers the base layer known as the LAMP setup - pairing up with Apache, MySQL, along with one of PHP, Python, or Perl. Its ability to manage tasks efficiently, handle memory wisely, coordinate network traffic, and support versatile file systems lets strong web servers operate smoothly even on basic machines. When tied together, an open-source operating system joins forces with freely available server and data tools, opening doors once locked behind costly, closed software. This shift rewired how online services are built, letting smaller teams and solo builders create what used to demand big budgets.

People Choices Growth

Out there, Linus Torvalds runs the core of the kernel while others handle pieces - networking here, drivers there. Work shows up openly on the Linux Kernel Mailing List, a place full of patch exchanges, feedback threads, arguments left in plain view.

Backed by big names like Google, Microsoft, Intel, IBM, Red Hat, and Samsung since its start in 2007, the Linux Foundation offers structure without steering tech choices. What keeps this huge software effort strong is clear rules, openness, while companies work together - yet go their own way on design.

Part D - Open Source vs Proprietary (Critical Analysis)

The following table compares the Linux Kernel against its closest proprietary alternatives - Microsoft Windows Server and Apple macOS - across six key dimensions.

Dimension	Linux Kernel (Open Source)	Proprietary Alternative (Windows / macOS)
Cost	Free of charge; GPL v2 source freely available to anyone, forever.	Windows Server: thousands of dollars per licence. macOS: tied to Apple hardware purchase. No perpetual free tier.
Security (Code Audit)	Source code is publicly readable. Any researcher, government, or developer can audit	Source is closed. Users must trust vendor security disclosures alone; independent

	every line for vulnerabilities independently.	third-party audit is impossible.
Support & Reliability	Community forums, mailing lists, Red Hat / SUSE enterprise support contracts available. Powers 90%+ of the world's servers reliably.	Official vendor SLAs available at high cost. Enterprise contracts required for production-level guarantees.
Freedom to Modify	Fully modifiable under GPL v2. Entire distributions (Ubuntu, Android, Chrome OS) are built on this freedom.	Cannot modify the OS kernel. Users are bound to the vendor's design decisions and update schedule permanently.
Community vs Corporate Control	Governed by Linus Torvalds and the Linux Foundation; thousands of contributors; no single corporation controls direction.	Single corporations control all design decisions, update timelines, and end-of-life schedules unilaterally.

Overall Verdict	Best choice for servers, development, cloud, embedded systems. Unmatched reliability, security transparency, and cost.	Suitable where specific proprietary software ecosystems (e.g., Microsoft Office suite or macOS creative apps) are mandatory.

Overall Verdict and Recommendation

Looking back at how the Linux Kernel grew - through decades of code, licensing choices, design shifts, and surrounding tools - trust in it feels natural when setting up servers, running clouds, building software, or working on small devices. Much of today's online world runs on Linux not simply due to price, but because it performs better under pressure. With open access to every line of code, fast fixes from users worldwide, and clear visibility into inner workings, its safety setup beats many secret commercial options. Low expense paired with high adaptability has sparked waves of invention that locked-down systems rarely match.

Few holdouts remain, like design apps locked into Apple's system or companies built around Microsoft’s directory service. Still, those gaps narrow - Linux grows stronger, web apps spread, lessening the need to stick with one setup.

Starting anywhere helps - fixing parts of the code or building drivers opens doors in the Linux Kernel world. Tiny changes might reach countless people, so jumping in feels useful, even satisfying.

5. Shell Script Tasks

Five shell scripts accompany this report. Each is documented below with its full code, explanation of purpose, and the shell scripting concepts it demonstrates. All scripts are designed to run on any standard Linux system with Bash installed.

● Script 1 - System Identity Report

This script acts as a welcome screen that introduces the Linux system. It displays the distribution name, kernel version, current user, home directory, system uptime, current date and time, and a note about the open-source licence covering the OS.

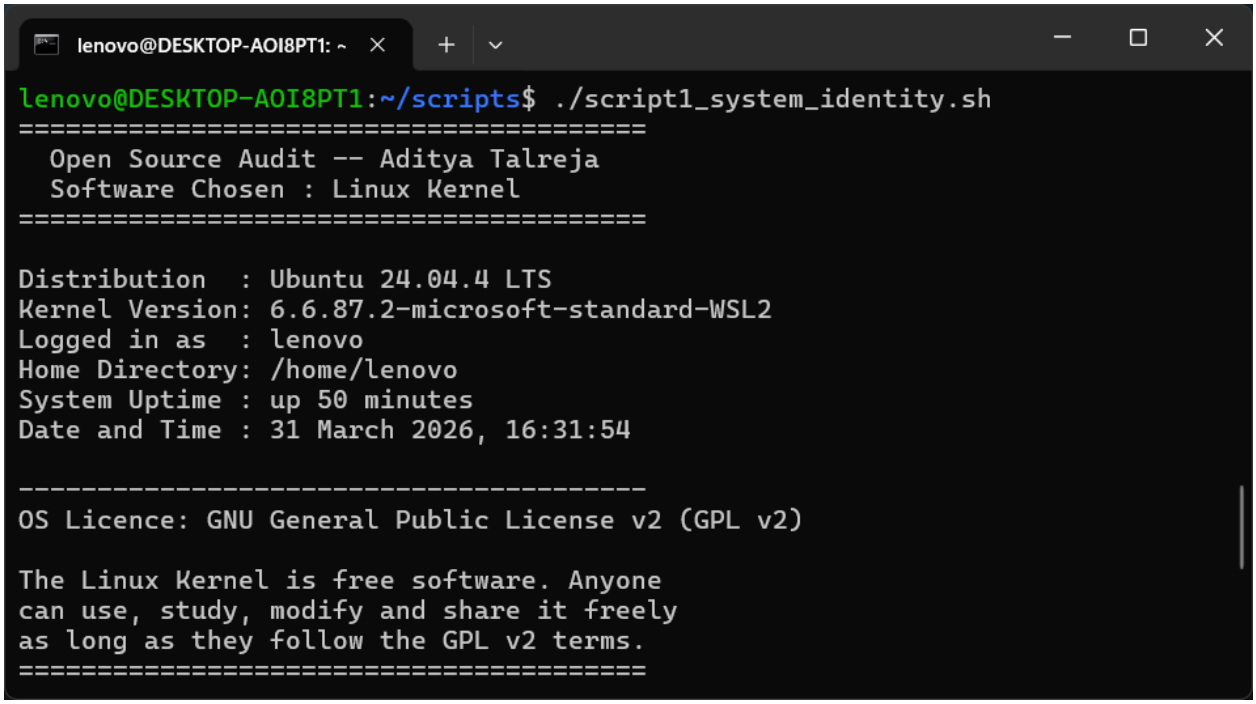
Concepts Used

- Variables -- storing command output and static text in named variables
- Command substitution with $() -- running a command and capturing its output
- echo -- formatted terminal output with labels
- uname, whoami, uptime, date -- standard Linux system information commands

Script Code

```
#!/bin/bash
# Script 1 - System Identity Report
# Author: Aditya Talreja | Reg No: 24BCE10891
# Course: Open Source Software | Slot: E11
# What this does: prints basic info about the system like a welcome screen
# I used this to understand how to pull live system data using variables
# storing my name and software choice as variables
STUDENT_NAME="[Your Name]"
SOFTWARE="Linux Kernel"
# using command substitution to grab system info
# uname -r gives kernel version, whoami gives current user
```

```
KERNEL=$(uname -r)
USERNAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
# date with a custom format so it looks readable
TODAY=$(date '+%d %B %Y, %H:%M:%S')
# getting distro name from os-release file
# had to use grep and cut to extract just the name part
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')
# the linux kernel is covered under GPL v2
LICENCE="GNU General Public License v2 (GPL v2)"
echo "========================================"
echo "  Open Source Audit -- $STUDENT_NAME"
echo "  Software Chosen : $SOFTWARE"
echo "========================================"
echo ""
echo "Distribution  : $DISTRO"
echo "Kernel Version: $KERNEL"
echo "Logged in as  : $USERNAME"
echo "Home Directory: $HOME_DIR"
echo "System Uptime : $UPTIME"
echo "Date and Time : $TODAY"
echo ""
echo "----------------------------------------"
echo "OS Licence: $LICENCE"
echo ""
echo "The Linux Kernel is free software. Anyone"
echo "can use, study, modify and share it freely"
echo "as long as they follow the GPL v2 terms."
echo "========================================"
```



- ## Script 2 - FOSS Package Inspector

This script checks whether a specified package is installed, displays its version and licence information, and uses a case statement to print a philosophical note about the package. It accepts the package name as a command-line argument.

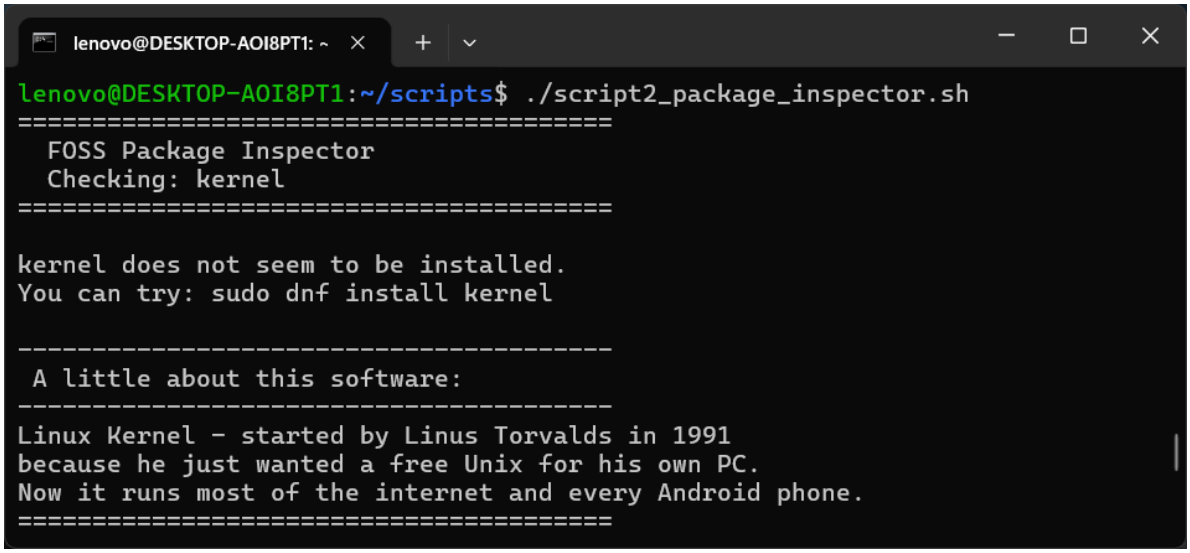
Concepts Used

- if-then-else -- conditional execution based on whether a package is installed
- case statement -- pattern matching to select a message based on package name
- rpm -qi -- querying installed package metadata
- pipe with grep -- filtering command output for specific fields

Script Code

```
#!/bin/bash
# Script 2 - FOSS Package Inspector
# Author: Aditya Talreja | Reg No: 24BCE10891
# Course: Open Source Software | Slot: E11
# Checks if a package is installed and shows some info about it
# also prints a short note about the philosophy behind each tool
# taking package name as input, defaulting to kernel if nothing is given
PACKAGE=${1:-"kernel"}
echo "========================================"
echo "  FOSS Package Inspector"
echo "  Checking: $PACKAGE"
echo "========================================"
echo ""
# checking if the package exists using rpm -q
# redirecting output to /dev/null so we only see our own messages
if rpm -q "$PACKAGE" &>/dev/null; then
       echo "$PACKAGE is installed on this system."
       echo ""
       # pulling version, licence and summary from rpm info
       # piping through grep to only show the fields I care about
       rpm -qi "$PACKAGE" | grep -E "^Version|^License|^Summary"
else
       echo "$PACKAGE does not seem to be installed."
       echo "You can try: sudo dnf install $PACKAGE"
fi
echo ""
echo "----------------------------------------"
echo " A little about this software:"
echo "----------------------------------------"
# case statement to print a short note based on which package was checked
# I added a few common ones and the ones from the assignment list
case "$PACKAGE" in
       kernel)
       echo "Linux Kernel - started by Linus Torvalds in 1991"
       echo "because he just wanted a free Unix for his own PC."
       echo "Now it runs most of the internet and every Android phone."
       ;;
```

```
        httpd | apache2)
        echo "Apache HTTP Server - one of the first open web servers."
        echo "Played a huge role in making the web accessible to everyone."
        ;;
        mysql | mariadb)
        echo "MySQL - a database used in millions of apps worldwide."
        echo "Interesting dual licence model - GPL and commercial."
        ;;
        git)
        echo "Git - Torvalds built this in about 10 days after BitKeeper"
        echo "revoked free access. Shows what frustration can create."
        ;;
        vlc)
        echo "VLC - built by students at a French university."
        echo "They just wanted to stream video on campus. Now everyone uses it."
        ;;
        firefox)
        echo "Firefox - Mozilla's fight to keep the web open."
        echo "Important alternative when IE was dominating everything."
        ;;
        *)
        echo "$PACKAGE is part of the open source world."
        echo "Most of what we use daily is built on free software like this."
        ;;
esac
echo "========================================"
```



- ## Script 3 - Disk and Permission Auditor

This script loops through a list of important system directories and reports how much space each uses, along with the owner, group, and permission bits. It also checks for the running kernel's module directory specifically.

Concepts Used

- for loop -- iterating over an array of directory paths
- if [-d] -- checking whether a path exists and is a directory
- ls -ld -- listing directory metadata (permissions, owner, group)
- du -sh -- reporting disk usage in human-readable format
- awk -- extracting specific fields from command output

Script Code

```
#!/bin/bash
# Script 3 - Disk and Permission Auditor
# Author: Aditya Talreja | Reg No: 24BCE10891
# Course: Open Source Software | Slot: E11
# Goes through some important Linux directories and prints
# their size and who owns them / what permissions they have
# list of directories I want to check
# these are the main ones that matter on any Linux system
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/boot")
echo "========================================"
echo "  Directory Audit Report"
echo "========================================"
echo ""
printf "%-18s %-22s %s\n" "Directory" "Permissions/Owner" "Size"
echo "----------------------------------------"
# looping through each directory in the list
for DIR in "${DIRS[@]}"; do
       # first checking if the directory actually exists
       if [ -d "$DIR" ]; then
       # ls -ld gives the long listing for the directory itself
       # awk pulls out just the permissions, owner, and group (fields 1, 3, 4)
```

```
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # du -sh gives size in human readable format
        # 2>/dev/null suppresses permission errors in places like /home
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-18s %-22s %s\n" "$DIR" "$PERMS" "$SIZE"
        else
        echo "$DIR -- not found on this system"
        fi
done
echo ""
echo "----------------------------------------"
echo " Checking Linux Kernel module directory:"
echo "----------------------------------------"
# uname -r gives the running kernel version
# using that to build the path to the modules folder
KVER=$(uname -r)
KMOD_DIR="/lib/modules/$KVER"
if [ -d "$KMOD_DIR" ]; then
        KPERMS=$(ls -ld "$KMOD_DIR" | awk '{print $1, $3, $4}')
        KSIZE=$(du -sh "$KMOD_DIR" 2>/dev/null | cut -f1)
        echo "Path          : $KMOD_DIR"
        echo "Permissions : $KPERMS"
        echo "Size          : $KSIZE"
else
        echo "Module directory not found for kernel $KVER"
fi
echo "========================================"
```

```
lenovo@DESKTOP-AOI8PT1:~/scripts$ ./script3_disk_auditor.sh
========================================
  Directory Audit Report
========================================

Directory          Permissions/Owner      Size
----------------------------------------
/etc               drwxr-xr-x root root   4.3M
/var/log           drwxrwxr-x root syslog 17M
/home              drwxr-xr-x root root   48K
/usr/bin           drwxr-xr-x root root   95M
/tmp               drwxrwxrwt root root   20K
/boot              drwxr-xr-x root root   4.0K

----------------------------------------
 Checking Linux Kernel module directory:
----------------------------------------
Path        : /lib/modules/6.6.87.2-microsoft-standard-WSL2
Permissions : drwxr-xr-x root root
Size        : 186M
========================================
```

- ## Script 4 - Log File Analyzer

This script reads a log file line by line, counts how many lines contain a specified keyword, and prints a summary along with the last five matching lines. It accepts the log file path and keyword as command-line arguments.

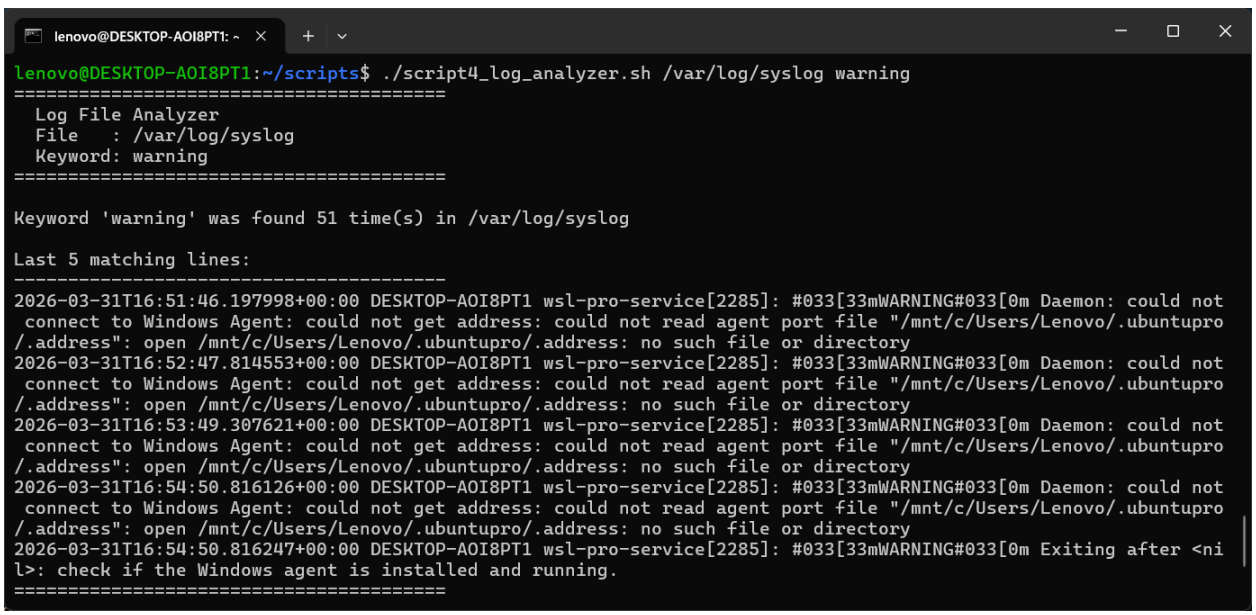
Concepts Used

- Command-line arguments ($1, $2) -- accepting input at runtime
- while IFS= read -r -- reading a file line by line safely
- if-then inside a loop -- conditional logic applied per line
- Counter variable -- incrementing a count with $(( ))
- grep with -i and tail -- case-insensitive search and last-N-lines display

Script Code

```
#!/bin/bash
# Script 4 - Log File Analyzer
# Author: Aditya Talreja | Reg No: 24BCE10891
# Course: Open Source Software | Slot: E11
# Reads a log file line by line and counts how many times
# a keyword appears, then shows the last few matching lines
# Usage: ./script4_log_analyzer.sh /path/to/logfile [keyword]
# Example: ./script4_log_analyzer.sh /var/log/messages error
# $1 is the log file path passed when running the script
LOGFILE=$1
# $2 is the keyword, defaulting to "error" if not given
KEYWORD=${2:-"error"}
# this counter will track how many matches we find
COUNT=0
# making sure the user actually gave a file path
if [ -z "$LOGFILE" ]; then
       echo "Please provide a log file path."
       echo "Usage: $0 <logfile> [keyword]"
       exit 1
fi
# checking the file exists before trying to read it
if [ ! -f "$LOGFILE" ]; then
       echo "Error: $LOGFILE not found."
       exit 1
fi
# if the file is there but completely empty, no point continuing
if [ ! -s "$LOGFILE" ]; then
       echo "The file $LOGFILE is empty, nothing to search."
       exit 0
fi
echo "========================================"
echo "  Log File Analyzer"
echo "  File    : $LOGFILE"
echo "  Keyword: $KEYWORD"
echo "========================================"
echo ""
# reading the file line by line using while read
```

```
# IFS= and -r make sure spacing and backslashes are preserved correctly
while IFS= read -r LINE; do
       # checking each line for the keyword, -i makes it case insensitive
       if echo "$LINE" | grep -iq "$KEYWORD"; then
       COUNT=$((COUNT + 1))    # incrementing counter when match found
       fi
done < "$LOGFILE"
# printing the final count
echo "Keyword '$KEYWORD' was found $COUNT time(s) in $LOGFILE"
echo ""
# showing the last 5 lines that matched so you can see context
echo "Last 5 matching lines:"
echo "----------------------------------------"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
echo "========================================"
```



- ## Script 5 -- The Open Source Manifesto Generator

This script interactively asks the user three questions, composes a personalised open-source philosophy statement from their answers, saves it to a text file, and displays it. It is the most creative of the five scripts.

Concepts Used

- read -p -- prompting the user for interactive input
- String building -- combining variables and fixed text into a paragraph
- Writing to a file with > and >> -- creating and appending to output files
- date command -- embedding the current date in output
- cat -- displaying the generated file contents to the terminal

Script Code

```
#!/bin/bash

# Script 5 - Open Source Manifesto Generator

# Author: Aditya Talreja | Reg No: 24BCE10891

# Course: Open Source Software | Slot: E11

# Asks you 3 questions and writes a short personal statement

# about open source based on your answers, saves it to a file

echo "========================================"

echo "  Open Source Manifesto Generator"

echo "========================================"

echo ""

echo "Answer 3 quick questions and I'll write"

echo "your personal open source statement."

echo ""

# read -p shows the question and waits for user to type something

read -p "1. One open-source tool you use every day: " TOOL

read -p "2. One word that 'freedom' means to you: " FREEDOM

read -p "3. One thing you'd build and share for free: " BUILD

# grabbing today's date for the file

TODAY=$(date '+%d %B %Y')

# output file named after the current user
```

```
OUTFILE="manifesto_$(whoami).txt"

# writing the header first, > creates/overwrites the file

echo "========================================" > "$OUTFILE"

echo "  MY OPEN SOURCE MANIFESTO"              >> "$OUTFILE"

echo "  Written on: $TODAY"                    >> "$OUTFILE"

echo "========================================" >> "$OUTFILE"

echo ""                                         >> "$OUTFILE"

# building the paragraph using the answers

# >> appends each line without overwriting what's already there

echo "I use $TOOL every single day without paying"       >> "$OUTFILE"

echo "for it. Someone built it, and then gave it away."  >> "$OUTFILE"

echo "That's not normal -- that's a choice. A choice"    >> "$OUTFILE"

echo "that says knowledge should belong to everyone."    >> "$OUTFILE"

echo ""                                                  >> "$OUTFILE"

echo "Freedom, to me, means $FREEDOM. And in software"  >> "$OUTFILE"

echo "that means being able to look inside the tools"    >> "$OUTFILE"

echo "I depend on -- not just use them blindly."         >> "$OUTFILE"

echo ""                                                  >> "$OUTFILE"

echo "Linus Torvalds just wanted a free OS for his PC."  >> "$OUTFILE"

echo "He shared it anyway. Now it runs the world."       >> "$OUTFILE"

echo "If I ever build $BUILD, I'll share it too."        >> "$OUTFILE"

echo "Because that's how I got everything I have."       >> "$OUTFILE"

echo ""                                                  >> "$OUTFILE"

echo "-- $(whoami), $TODAY"                              >> "$OUTFILE"

echo "========================================" >> "$OUTFILE"

echo ""

echo "Saved to: $OUTFILE"
```

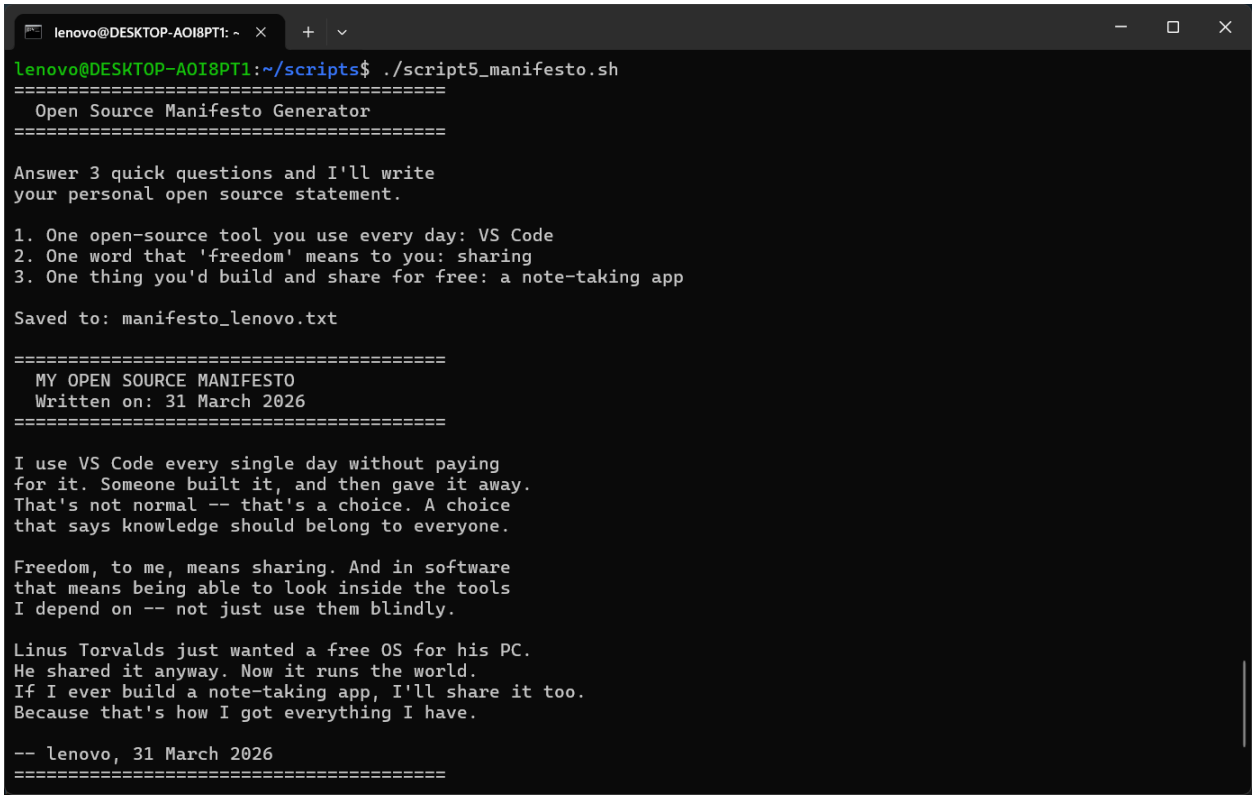
```
echo ""

# alias concept: you could run this script faster with an alias like:

# alias manifesto='bash ~/script5_manifesto.sh'

# that's what aliases do - short names for longer commands

cat "$OUTFILE"
```



6. Conclusion

This journey began with a student tinkering in 1991. Today, Linux runs beneath nearly every digital system. Its power isn’t just code, it's the idea of sharing, built through countless acts of cooperation across borders and time zones.

The kernel thrives on freedom and transparency. When people can see and modify how things work, progress follows. Open collaboration outperforms closed systems, and trust grows through visibility.

The accompanying shell scripts reflect this: clear, commented, and fully traceable. Users can understand, adjust, and improve them.

Linux shows what shared effort can achieve. One patch at a time, fixes, reviews, and contributions keep it alive. Progress comes from showing up again and again.

7. References

- Torvalds, L. (1991). Original Linux announcement -- comp.os.minix newsgroup.
- GNU Project -- The Free Software Definition: gnu.org/philosophy/free-sw.html
- Open Source Initiative -- The Open Source Definition: opensource.org/osd
- The Linux Kernel Archives: kernel.org
- Raymond, E.S. -- The Cathedral and the Bazaar: catb.org/~esr/writings/cathedral-bazaar
- SPDX Licence List -- GPL v2: spdx.org/licenses/GPL-2.0-only.html
- The Linux Foundation: linuxfoundation.org
- TOP500 Supercomputer List: top500.org
- Shotts, W. -- The Linux Command Line: linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual